Báo cáo tích hợp Chương 6 — Specs → Kripke → Code → CWE/CVE → Ch1-5

Đặc tả: atm_fuzzing_verification

Mục lục

1. Đặc tả yêu cầu (Specification) 2

2. Chương 1 — Kiểm chứng hình thức (Kripke/CTL) 4

2.1. Sơ đồ chuyển trạng thái 4

3. Chương 2 — Mã C sinh ra & Static Checker 4

3.1. Python wrap cho mã C (Ch2/3/5) — detect → wrap ctypes → sinh test → CWE 5

4. Chương 3 — Trích xuất AST/điều kiện & SMT (Z3) 8

4.0a. CFG (Control Flow Graph) từng hàm Python 8

4.0b. Call Graph (toàn bộ hàm Python sinh từ đặc tả) 9

4.0d. CFG + Call Graph của mã C — dựng QUA lớp Python wrap 9

4.1. Test case SINH TỰ ĐỘNG từ SMT (symbolic execution) 10

5. Chương 4 — Dynamic Fuzzing (hàm Python sinh ra) 10

5.0. Tham số TỰ ĐỘNG DETECT (theo tên biến) 10

5.1. Mutation fuzzing (seed tự sinh từ tham số đã detect) 11

5.2. Coverage tracking (line/branch, 1 lần chạy đại diện) 11

5.3. Test case sinh từ TẤT CẢ kiểu fuzzing (Python trực tiếp; C qua Python wrap) 11

6. Chương 5 — Phân loại LUỒNG Lập trình C/C++ an toàn 15

7. Chương 6 — Phân loại CWE/CVE tổng hợp 15

8. Test case truy vết từ đặc tả (tài liệu hóa) 16

1. Đặc tả yêu cầu (Specification)

Đặc tả hệ thống ATM phục vụ kiểm chứng hình thức và fuzzing, tập trung vào xác thực PIN, quản lý số lần nhập sai, khóa tài khoản.

Mã yêu cầu	Loại	Ưu tiên	Nội dung
REQ-1	functional	HIGH	Hệ thống chỉ chấp nhận PIN gồm đúng 4 chữ số từ 0 đến 9; từ chối chuỗi rỗng, sai độ dài hoặc chứa ký tự không phải số.
REQ-2	functional	HIGH	Khi PIN hợp lệ và đúng với PIN mô phỏng đã lưu, hệ thống chuyển phiên từ IDLE sang AUTHENTICATED.
REQ-3	safety	HIGH	Khi PIN sai hoặc không hợp lệ, hệ thống từ chối xác thực và tăng số lần thử sai. Nếu sai lần thứ nhất, hệ thống chuyển sang PIN_FAILED_1; nếu sai lần thứ hai, chuyển sang PIN_FAILED_2; nếu sai lần thứ ba, chuyển sang LOCKED và không cho phép tiếp tục xác thực trong phiên đó.
REQ-4	safety	HIGH	Khi số lần nhập PIN sai đạt 3, hệ thống chuyển sang LOCKED và không cho phép xác thực hoặc giao dịch tiếp tục trong phiên đó.
REQ-5	functional	MEDIUM	Xây dựng hàm thử nghiệm có nhánh kiểm tra giá trị PIN quản trị giả lập để đánh giá khả năng tìm đầu vào bằng concolic fuzzing.
REQ-6	memory	HIGH	Kiểm tra tính an toàn của bộ đệm nhận chuỗi PIN, phát hiện nguy cơ tràn bộ đệm khi đầu vào vượt giới hạn.

Mã yêu cầu	Loại	Ưu tiên	Nội dung
NFR-1	non_functional	HIGH	Mô hình ATM phải cho phép kiểm tra tự động các trạng thái và chuyển trạng thái bằng công cụ kiểm chứng
NFR-2	non_functional	HIGH	Bộ sinh đầu vào fuzzing phải có khả năng tạo các giá trị hợp lệ và bất thường cho các tham số đầu vào
NFR-3	non_functional	HIGH	Hệ thống phải ghi nhận được các đầu vào làm vi phạm thuộc tính an toàn để phục vụ phân tích counterexample
NFR-4	non_functional	MEDIUM	Việc kiểm chứng phải có khả năng lặp lại với cùng seed của bộ sinh fuzzing
NFR-5	non_functional	HIGH	Hệ thống phải ghi nhận kết quả kiểm chứng và kết quả fuzzing, bao gồm đầu vào gây lỗi và trạng thái liên quan, để có thể tái hiện và phân tích lỗi.
CON-1	constraint	HIGH	PIN phải gồm đúng 4 ký tự số từ 0 đến 9; PIN sai độ dài hoặc chứa ký tự không phải chữ số đều phải bị từ chối.
CON-2	constraint	HIGH	Số lần nhập sai liên tiếp được theo dõi qua hai trạng thái trung gian PIN_FAILED_1 và PIN_FAILED_2.
CON-3	constraint	HIGH	Khi nhập sai liên tiếp lần thứ 3 (từ PIN_FAILED_2), hệ thống chuyển sang trạng thái LOCKED và khóa thẻ.
CON-4	constraint	HIGH	Tài khoản ở trạng thái LOCKED tuyệt đối không được phép chuyển sang trạng thái AUTHENTICATED.

Mã yêu cầu	Loại	Ưu tiên	Nội dung
CON-5	constraint	HIGH	Tài khoản không được đồng thời ở hai trạng thái AUTHENTICATED và LOCKED

2. Chương 1 — Kiểm chứng hình thức (Kripke/CTL)

Trạng thái deadlock: không có

Công thức CTL	Kết quả	Trạng thái thỏa
AG(!(LOCKED && AUTHENTICATED))	ĐÚNG	AUTHENTICATED, IDLE, LOCKED, PIN_FAILED_1, PIN_FAILED_2
EF(LOCKED)	ĐÚNG	AUTHENTICATED, IDLE, LOCKED, PIN_FAILED_1, PIN_FAILED_2
EF(AUTHENTICATED)	ĐÚNG	AUTHENTICATED, IDLE, PIN_FAILED_1, PIN_FAILED_2

Mutual exclusion: OK (trạng thái nguy hiểm: không có)

2.1. Sơ đồ chuyển trạng thái

Sơ đồ: Mô hình chính: atm_fuzzing_verification

Mã TikZ (LaTeX) — dán trực tiếp vào tài liệu LaTeX có \usepackage{tikz} để biên dịch thành hình vector, thay cho ảnh tĩnh:

```
% Mô hình chính: atm_fuzzing_verification
\begin{tikzpicture}[
every node/.style={draw, circle, inner sep=2pt,
minimum size=8mm, align=center, font=\scriptsize},
->, >=stealth, thick, auto
]
\node[fill=blue!25] (nd_IDLE) at (-0.77,0.30) {IDLE\\{\tiny IDLE}};
\node[fill=gray!15] (nd_AUTHENTICATED) at (1.62,0.03) {AUTHENTICATED\\{\tiny AUTHENTICATED,PIN\_VALID}};
\node[fill=gray!15] (nd_PIN_FAILED_2) at (0.05,-1.03) {PIN\_FAILED\_2\\{\tiny PIN\_FAILED\_2,PIN\_INVALID}};
\node[fill=gray!15] (nd_PIN_FAILED_1) at (-1.27,-1.51) {PIN\_FAILED\_1\\{\tiny PIN\_FAILED\_1,PIN\_INVALID}};
\node[fill=gray!15] (nd_LOCKED) at (0.37,2.20) {LOCKED\\{\tiny LOCKED}};
\draw (nd_IDLE) -- (nd_PIN_FAILED_1);
\draw (nd_IDLE) -- (nd_AUTHENTICATED);
\draw (nd_AUTHENTICATED) -- (nd_IDLE);
\draw (nd_PIN_FAILED_2) -- (nd_AUTHENTICATED);
\draw (nd_PIN_FAILED_2) -- (nd_LOCKED);
\draw (nd_PIN_FAILED_1) -- (nd_PIN_FAILED_2);
\draw (nd_PIN_FAILED_1) -- (nd_AUTHENTICATED);
\draw (nd_LOCKED) -- (nd_LOCKED);
\end{tikzpicture}
```

3. Chương 2 — Mã C sinh ra & Static Checker

Mã C sinh từ đặc tả

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* sinh từ đặc tả 'atm_fuzzing_verification', hàm 'parse_atm_pin', kind='buffer_copy' */
void parse_atm_pin(const char *raw_pin) {
char buffer[16];
strcpy(buffer, raw_pin); /* sinh từ đặc tả: buffer-overflow */
```

```
printf("%s\n", buffer);
}
```

Dòng	Mức độ	Loại lỗi	Mô tả
8	HIGH	dangerous-function	Sử dụng hàm không an toàn 'strcpy()' — dễ gây buffer overflow. Goi y: dung strncpy thay the.

3.1. Python wrap cho mã C (Ch2/3/5) — detect → wrap ctypes → sinh test → CWE

Mã C sinh từ đặc tả được bọc bằng lớp Python (ctypes, core.c_wrapper): các hàm wrap được tự động detect (hoặc do người học chọn từ danh sách dạng pytest), chạy trong tiến trình con cô lập để crash của mã C không làm sập công cụ. Nhờ đó Chương 2 xác nhận lỗi bộ nhớ bằng THỰC THI (không chỉ heuristic tĩnh), Chương 3 dựng CFG/Call Graph có tô màu coverage, Chương 5 so sánh mã không an toàn/an toàn.

Kế hoạch wrap (danh sách pytest)

```
Kế hoạch wrap (C, module 'atm_fuzzing_verification_wrap'): 1/1 hàm wrap được, 1 hàm được chọn.
☑ test_atm_fuzzing_verification_wrap.py::test_wrap_parse_atm_pin
C: void parse_atm_pin(const char * raw_pin)
tham số: raw_pin:str; trả về: void
```

Build thư viện wrap: THÀNH CÔNG; số test case sinh ra: 15, trong đó 12 test kích hoạt crash.

Hàm (qua wrap)	Loại lỗi động	CWE
parse_atm_pin() [wrap]	c-hang	CWE-835 — Loop with Unreachable Exit Condition ('Infinite Loop')
parse_atm_pin() [wrap]	c-out-of-bounds	CWE-787 — Out-of-bounds Write

Module Python wrap sinh tự động (ctypes)

```
"""Python wrap (ctypes) cho mã C — SINH TỰ ĐỘNG bởi sec_lab_framework (core/c_wrapper.py).

Hàm được wrap: parse_atm_pin.
Dùng:
import atm_fuzzing_verification_wrap as w
w.parse_atm_pin(...) # gọi TRỰC TIẾP (C crash => Python crash)
w.call_isolated("parse_atm_pin", ...) # gọi trong TIẾN TRÌNH CON (an toàn khi C crash/treo)
Nếu chưa có thư viện `libatm_fuzzing_verification_wrap.dll`, module tự biên dịch từ `atm_fuzzing_verification_shim.c` +
`sl_target.c` bằng `gcc`.
"""
import json, os, subprocess, sys, tempfile

_HERE = os.path.dirname(os.path.abspath(__file__))
_LIB_PATH = os.path.join(_HERE, 'libatm_fuzzing_verification_wrap.dll')
_SHIM = os.path.join(_HERE, 'atm_fuzzing_verification_shim.c')
_LANG = 'c'
SIGNATURES = {'parse_atm_pin': {'params': [{'kind': 'str', 'ctype': 'c_char', 'const': True}],
'ret': {'kind': 'void', 'ctype': '', 'const': False},
'names': ['raw_pin']}}

# ---- runtime dùng chung (marshal đối số, worker cô lập) — cùng mã với framework ----
import ctypes, json, os, sys

_libc = None

def sl_alloc(data):
"""Cap phat bo dem bang malloc CUA C (dung kich thuoc, khong them dem) va chep `data` vao.
Nho vay tran/doc ngoai bien cua ma C cham vao heap that (ASan dat redzone; heap hong -> crash that),
thay vi bo dem do Python quan ly. Khong free: worker song ngan; callee co the tu free."""
```

```
global _libc
if _libc is None:
_libc = ctypes.CDLL(None) if os.name != "nt" else ctypes.cdll.msvcrt
_libc.malloc.restype = ctypes.c_void_p
_libc.malloc.argtypes = [ctypes.c_size_t]
p = _libc.malloc(max(len(data), 1))
if data:
ctypes.memmove(p, data, len(data))
return p

def sl_marshal_arg(spec, v, keep):
k = spec["kind"]
if k == "int":
return int(v)
if k == "float":
return float(v)
if k == "bool":
return bool(v)
if k in ("str", "bytes"):
if v is None:
return None
if isinstance(v, str):
b = v.encode("utf-8", "surrogateescape")
elif isinstance(v, (bytes, bytearray)):
b = bytes(v)
else:
b = str(v).encode("utf-8", "surrogateescape")
return sl_alloc(b + b"\0")
if k == "array":
if v is None:
return None
elem = getattr(ctypes, spec["ctype"])
lst = list(v)
arr = (elem * max(len(lst), 1))(*lst)
return sl_alloc(bytes(arr)[: len(lst) * ctypes.sizeof(elem)])
raise TypeError("kieu tham so khong ho tro: %r" % (k,))

def sl_argtype(spec):
k = spec["kind"]
if k in ("int", "float", "bool"):
return getattr(ctypes, spec["ctype"])
return ctypes.c_void_p

def sl_unmarshal_ret(spec, r):
k = spec["kind"]
if k == "void":
return None
if k == "str":
return ctypes.string_at(r).decode("utf-8", "replace") if r else None
if k == "ptr":
return int(r) if r else 0
return r

def sl_setup(lib, name, sig):
fn = getattr(lib, "sl_wrap_" + name)
fn.argtypes = [sl_argtype(p) for p in sig["params"]]
rk = sig["ret"]["kind"]
fn.restype = None if rk == "void" else (ctypes.c_void_p if rk in ("str", "ptr") else getattr(ctypes, sig["ret"]["ctype"]))
return fn

def sl_worker_main(job_path):
job = json.load(open(job_path, encoding="utf-8"))
lib = ctypes.CDLL(job["lib"])
fns = {}
want_cov = bool(job.get("want_cov")) and hasattr(lib, "sl_cov_ptr")
if want_cov:
lib.sl_cov_ptr.restype = ctypes.c_void_p
lib.sl_cov_reset.restype = None
n_probes = int(job.get("n_probes", 0))
out = open(job["out"], "a", encoding="utf-8")
```

```
off = int(job.get("offset", 0))
for j, call in enumerate(job["calls"]):
name = call["f"]
sig = job["sigs"][name]
try:
if name not in fns:
fns[name] = sl_setup(lib, name, sig)
keep = []
cargs = [sl_marshal_arg(sp, v, keep) for sp, v in zip(sig["params"], call["args"])]
except Exception as e:
out.write(json.dumps({"i": off + j, "status": "error", "detail": "marshal: %r" % (e,)}) + "\n")
out.flush()
continue
if want_cov:
lib.sl_cov_reset()
try:
r = fns[name](*cargs)
rec = {"i": off + j, "status": "ok", "ret": sl_unmarshal_ret(sig["ret"], r)}
except Exception as e:
rec = {"i": off + j, "status": "error", "detail": repr(e)}
if want_cov:
raw = ctypes.string_at(lib.sl_cov_ptr(), n_probes)
rec["cov"] = [q for q, b in enumerate(raw) if b]
out.write(json.dumps(rec) + "\n")
out.flush()
out.close()
return 0


def build_library(force=False):
"""Biên dịch lại thư viện từ shim (cần gcc/clang, hoặc g++/clang++ cho C++)."""
if os.path.exists(_LIB_PATH) and not force:
return _LIB_PATH
import shutil
cands = (["g++", "c++", "clang++"] if _LANG == "cpp" else ["gcc", "cc", "clang"])
cc = next((c for c in cands if shutil.which(c)), None)
if cc is None:
raise RuntimeError("Không tìm thấy trình biên dịch C/C++ để build thư viện wrap")
cmd = [cc, "-shared", "-fPIC", "-O0", "-g"] + (["-std=c++17"] if _LANG == "cpp" else []) + \
["-I", _HERE, "-o", _LIB_PATH, _SHIM]
r = subprocess.run(cmd, capture_output=True, text=True)
if r.returncode != 0:
raise RuntimeError("Biên dịch thất bại:\n" + r.stderr)
return _LIB_PATH


_lib = None
_fns = {}


def _fn(name):
global _lib
if _lib is None:
build_library()
_lib = ctypes.CDLL(_LIB_PATH)
if name not in _fns:
_fns[name] = sl_setup(_lib, name, SIGNATURES[name])
return _fns[name]


def call_direct(name, *args):
"""Gọi TRỰC TIẾP trong tiến trình hiện tại (nếu mã C crash thì Python cũng crash)."""
sig = SIGNATURES[name]
keep = []
cargs = [sl_marshal_arg(sp, v, keep) for sp, v in zip(sig["params"], args)]
return sl_unmarshal_ret(sig["ret"], _fn(name)(*cargs))


def call_isolated(name, *args, timeout=5.0):
"""Gọi trong TIẾN TRÌNH CON: trả về dict {"status": ok|crash|timeout|error, "ret": ..., "signal": ...}."""
```

```
build_library()
tmp = tempfile.mkdtemp(prefix="sl_call_")
try:
job = {"lib": _LIB_PATH, "sigs": SIGNATURES, "offset": 0, "want_cov": False, "n_probes": 0,
"out": os.path.join(tmp, "out.jsonl"), "calls": [{"f": name, "args": list(args)}]}
jp = os.path.join(tmp, "job.json")
with open(jp, "w", encoding="utf-8") as f:
json.dump(job, f)
open(job["out"], "w").close()
try:
p = subprocess.run([sys.executable, os.path.abspath(__file__), "--sl-worker", jp],
capture_output=True, text=True, timeout=timeout)
except subprocess.TimeoutExpired:
return {"status": "timeout", "ret": None, "signal": "TIMEOUT"}
lines = [l for l in open(job["out"], encoding="utf-8").read().splitlines() if l.strip()]
if lines:
return json.loads(lines[-1])
rc = p.returncode
sig = ""
if rc < 0:
import signal as _s
try:
sig = _s.Signals(-rc).name
except Exception:
sig = "SIG%d" % -rc
return {"status": "crash" if rc != 0 else "exit", "ret": None, "signal": sig,
"returncode": rc, "stderr": p.stderr[-500:]}
finally:
import shutil
shutil.rmtree(tmp, ignore_errors=True)


# ---- hàm Python wrap (mỗi hàm C một hàm Python cùng tên) ----
def parse_atm_pin(raw_pin):
"""C: void parse_atm_pin(const char * raw_pin) (dòng 6)

Tham số: raw_pin:str. Trả về: void."""
return call_direct('parse_atm_pin', raw_pin)

if __name__ == '__main__' and len(sys.argv) > 2 and sys.argv[1] == '--sl-worker':
sys.exit(sl_worker_main(sys.argv[2]))
```

4. Chương 3 — Trích xuất AST/điều kiện & SMT (Z3)

Hàm	Biến	Điều kiện	Vòng lặp	Độ phức tạp McCabe	Độ sâu lồng nhau
parse_atm_pin	['buffer']	[]	[]	1	1
atm_pin_retry_ratio	[]	[]	[]	1	1
atm_admin_override_pin	[]	['pin_a == 7421', 'pin_b == 3390']	[]	3	3

4.0a. CFG (Control Flow Graph) từng hàm Python

Hàm	Tóm tắt CFG
atm_pin_retry_ratio	3 khối lệnh (basic block), 2 cạnh -> độ phức tạp McCabe (từ CFG) ≈ 1

Hàm	Tóm tắt CFG
atm_admin_override_pin	8 khối lệnh (basic block), 9 cạnh -> độ phức tạp McCabe (từ CFG) ≈ 3

4.0b. Call Graph (toàn bộ hàm Python sinh từ đặc tả)

3 hàm, 1 lệnh gọi: atm_admin_override_pin->ValueError

4.0d. CFG + Call Graph của mã C — dựng QUA lớp Python wrap

Node xanh = đã được thực thi khi chạy các test sinh tự động qua wrap; xám + viền đỏ = CHƯA được phủ. Nhánh py:<hàm> là lớp Python wrap gọi vào hàm C.

Hàm C	Tóm tắt CFG (qua wrap)
parse_atm_pin	3 khối lệnh (basic block), 2 cạnh -> độ phức tạp McCabe (từ CFG) ≈ 1; coverage khi chạy qua wrap: 1/1 khối (100%)

Sơ đồ: CFG parse_atm_pin()

Mã TikZ (LaTeX) — dán trực tiếp vào tài liệu LaTeX có \usepackage{tikz} để biên dịch thành hình vector, thay cho ảnh tĩnh:

```
% CFG parse_atm_pin() (mã C qua Python wrap)
\begin{tikzpicture}[
every node/.style={draw, rectangle, rounded corners, inner sep=2pt,
minimum size=8mm, align=center, font=\scriptsize},
->, >=stealth, thick, auto
]
\node[fill=green!30, draw=green!50!black, line width=1pt] (nd_B0) at (0.00,0.00) {ENTRY};
\node[fill=orange!35, draw=green!50!black, line width=1pt] (nd_EXIT) at (0.00,-4.48) {EXIT};
\node[fill=cyan!20, draw=green!50!black, line width=1pt] (nd_B1) at (0.00,-2.24) {char buffer[16];
strcpy(buffer, raw\_pin);
printf("\%s\textbackslash{}n", buffer);};
\draw (nd_B0) -- (nd_B1);
\draw (nd_B1) -- (nd_EXIT);
\end{tikzpicture}
```

Sơ đồ: Call Graph mã C

Mã TikZ (LaTeX) — dán trực tiếp vào tài liệu LaTeX có \usepackage{tikz} để biên dịch thành hình vector, thay cho ảnh tĩnh:

```
% Call Graph mã C (lớp Python wrap)
\begin{tikzpicture}[
every node/.style={draw, rectangle, rounded corners, inner sep=2pt,
minimum size=8mm, align=center, font=\scriptsize},
->, >=stealth, thick, auto
]
\node[fill=cyan!25, draw=green!50!black, line width=1pt] (nd_parse_atm_pin) at (-0.06,0.33) {parse\_atm\_pin};
\node[fill=gray!25] (nd_strcpy) at (-2.00,-0.23) {strcpy};
\node[fill=gray!25] (nd_printf) at (1.95,0.51) {printf};
\node[fill=green!25, draw=green!50!black, line width=1pt] (nd_py_parse_atm_pin) at (0.12,-0.62) {py:parse\_atm\_pin};
\draw (nd_parse_atm_pin) -- (nd_strcpy);
\draw (nd_parse_atm_pin) -- (nd_printf);
\draw[dashed] (nd_py_parse_atm_pin) -- (nd_parse_atm_pin);
\end{tikzpicture}
```

1 hàm nội bộ, 2 hàm thư viện, 2 lệnh gọi: parse_atm_pin->strcpy, parse_atm_pin->printf

Kiểm tra	Kết quả SMT	Model (nếu SAT)
atm_auth_counter	sat	{'failed_attempts': 2000000001, 'max_retries': 0}

Kiểm tra	Kết quả SMT	Model (nếu SAT)
atm_auth_counter (int32 overflow?)	sat	{'failed_attempts': -1, 'max_retries': -2147483648}

4.1. Test case SINH TỰ ĐỘNG từ SMT (symbolic execution)

Bổ sung cho Chương 4 bên dưới: thay vì chỉ tìm 1 input gây crash, bộ sinh test này duyệt các nhánh if/while/for (trong giới hạn) của CHÍNH các hàm Python sinh từ đặc tả, dùng Z3 giải điều kiện đường đi để sinh 1 bộ test ĐẠT ĐỘ PHỦ NHÁNH (branch coverage) đầy đủ, tự động.
Hàm 'atm_pin_retry_ratio' — 1 test case, 0 đường đi UNSAT (không khả thi):

#	Input	Kết quả	Đường đi (path condition)
1	total_attempts=0, failed_attempts=0	NÉM ZeroDivisionError: division by zero	(đường đi thẳng, không rẽ nhánh)

Hàm 'atm_admin_override_pin' — 3 test case, 0 đường đi UNSAT (không khả thi):

#	Input	Kết quả	Đường đi (path condition)
1	pin_a=7421, pin_b=3390	NÉM ValueError: magic value reached	if pin_a == 7421 -> True; if pin_b == 3390 -> True
2	pin_a=7421, pin_b=3391	10812	if pin_a == 7421 -> True; if pin_b == 3390 -> False
3	pin_a=7422, pin_b=0	7422	if pin_a == 7421 -> False

5. Chương 4 — Dynamic Fuzzing (hàm Python sinh ra)

Hàm Python 'atm_pin_retry_ratio'

```
def atm_pin_retry_ratio(total_attempts, failed_attempts):
return total_attempts // failed_attempts
```

Hàm Python 'atm_admin_override_pin'

```
def atm_admin_override_pin(pin_a, pin_b):
if pin_a == 7421:
if pin_b == 3390:
raise ValueError('magic value reached')
return pin_a + pin_b
```

5.0. Tham số TỰ ĐỘNG DETECT (theo tên biến)

Hàm	Tham số : kiểu (đã detect)
atm_pin_retry_ratio	total_attempts:int, failed_attempts:int
atm_admin_override_pin	pin_a:int, pin_b:int

Hàm	Kết quả fuzzing (black-box / concolic)
atm_pin_retry_ratio	black-box: 11/300 lần crash

Hàm	Kết quả fuzzing (black-box / concolic)
atm_admin_override_pin	black-box: 0/300 lần crash | concolic: crash_input={'pin_a': 7421, 'pin_b': 3390} sau 3 lần thử

5.1. Mutation fuzzing (seed tự sinh từ tham số đã detect)

Hàm	Kết quả mutation fuzzing
atm_pin_retry_ratio	mutation: 81/200 lần crash (seed tự sinh: [(0, 0), (1, 1), (-1, -1)])
atm_admin_override_pin	mutation: 0/200 lần crash (seed tự sinh: [(0, 0), (1, 1), (-1, -1)])

5.2. Coverage tracking (line/branch, 1 lần chạy đại diện)

Hàm	Coverage
atm_pin_retry_ratio	Lines hit: 1 | Branches hit: 0 | Calls: ['atm_pin_retry_ratio'] | Exception: ZeroDivisionError: division by zero
atm_admin_override_pin	Lines hit: 2 | Branches hit: 1 | Calls: ['atm_admin_override_pin'] | Exception: None

5.3. Test case sinh từ TẤT CẢ kiểu fuzzing (Python trực tiếp; C qua Python wrap)

Mỗi kỹ thuật (boundary, black-box, mutation, coverage-guided, concolic) được chạy và ghi lại: giữ mỗi crash duy nhất cùng các input làm tăng coverage; xuất được thành pytest (nút 'Xuất GÓI pytest').
Hàm 'atm_pin_retry_ratio' (PYTHON): 22 test case, 8 kích hoạt lỗi. Coverage dòng của 'atm_pin_retry_ratio': 1/1 (100%), 0 cặp nhánh

#	Kỹ thuật	Input	Kết quả / lỗi	Ghi chú
1	Biên / giá trị đặc biệt	total_attempts=1.0, failed_attempts=0	EXCEPTION ZeroDivisionError	kích hoạt lỗi
2	Biên / giá trị đặc biệt	total_attempts=0, failed_attempts=0	EXCEPTION ZeroDivisionError	kích hoạt lỗi
3	Biên / giá trị đặc biệt	total_attempts=1.0, failed_attempts=1	1.0	+1 phần tử coverage mới
4	Biên / giá trị đặc biệt	total_attempts=0.0, failed_attempts=1	0.0	kết quả đại diện
5	Biên / giá trị đặc biệt	total_attempts=-0.0, failed_attempts=1	-0.0	kết quả đại diện
6	Biên / giá trị đặc biệt	total_attempts=-1.0, failed_attempts=1	-1.0	kết quả đại diện
7	Black-box (ngẫu nhiên)	total_attempts=1e+308, failed_attempts=0	EXCEPTION ZeroDivisionError	kích hoạt lỗi
8	Black-box (ngẫu nhiên)	total_attempts=22.48898459858779, failed_attempts=0	EXCEPTION ZeroDivisionError	kích hoạt lỗi

#	Kỹ thuật	Input	Kết quả / lỗi	Ghi chú
9	Black-box (ngẫu nhiên)	total_attempts=nan, failed_attempts=-1642	nan	kết quả đại diện
10	Black-box (ngẫu nhiên)	total_attempts=-55509.512847766746, failed_attempts=33	-1683.0	kết quả đại diện
11	Black-box (ngẫu nhiên)	total_attempts=1000000000.0, failed_attempts=175	5714285.0	kết quả đại diện
12	Black-box (ngẫu nhiên)	total_attempts=-467338.87908548093, failed_attempts=9370	-50.0	kết quả đại diện
13	Mutation (đột biến seed)	total_attempts=-0.0, failed_attempts=0	EXCEPTION ZeroDivisionError	kích hoạt lỗi
14	Mutation (đột biến seed)	total_attempts=1.0474234445716815, failed_attempts=0	EXCEPTION ZeroDivisionError	kích hoạt lỗi
15	Mutation (đột biến seed)	total_attempts=0.0, failed_attempts=2305843009213693953	0.0	kết quả đại diện
16	Mutation (đột biến seed)	total_attempts=1e+308, failed_attempts=1	1e+308	kết quả đại diện
17	Mutation (đột biến seed)	total_attempts=-1.9128507628962974, failed_attempts=1	-2.0	kết quả đại diện
18	Coverage-guided	total_attempts=-1.0, failed_attempts=0	EXCEPTION ZeroDivisionError	crash mới
19	Coverage-guided	total_attempts=-0.44574119565331194, failed_attempts=1	-1.0	kết quả đại diện
20	Coverage-guided	total_attempts=1.0, failed_attempts=2	0.0	kết quả đại diện
21	Coverage-guided	total_attempts=2.0, failed_attempts=1	2.0	kết quả đại diện
22	Concolic / white-box (SMT + hằng số)	total_attempts=0.0, failed_attempts=0	EXCEPTION ZeroDivisionError	kích hoạt lỗi

Hàm 'atm_admin_override_pin' (PYTHON): 17 test case, 1 kích hoạt lỗi. Coverage dòng của 'atm_admin_override_pin': 4/4 (100%), 4 cặp nhánh

#	Kỹ thuật	Input	Kết quả / lỗi	Ghi chú
1	Biên / giá trị đặc biệt	pin_a=1, pin_b=1	2	+3 phần tử coverage mới
2	Biên / giá trị đặc biệt	pin_a=7421, pin_b=1	7422	+3 phần tử coverage mới
3	Biên / giá trị đặc biệt	pin_a=0, pin_b=1	1	kết quả đại diện
4	Biên / giá trị đặc biệt	pin_a=2, pin_b=1	3	kết quả đại diện
5	Black-box (ngẫu nhiên)	pin_a=7, pin_b=106	113	kết quả đại diện
6	Black-box (ngẫu nhiên)	pin_a=77, pin_b=-6925	-6848	kết quả đại diện
7	Black-box (ngẫu nhiên)	pin_a=804833100429, pin_b=-9931	804833090498	kết quả đại diện

#	Kỹ thuật	Input	Kết quả / lỗi	Ghi chú
8	Black-box (ngẫu nhiên)	pin_a=185, pin_b=9	194	kết quả đại diện
9	Mutation (đột biến seed)	pin_a=7421, pin_b=0	7421	kết quả đại diện
10	Mutation (đột biến seed)	pin_a=3390, pin_b=1	3391	kết quả đại diện
11	Mutation (đột biến seed)	pin_a=1, pin_b=7421	7422	kết quả đại diện
12	Mutation (đột biến seed)	pin_a=1, pin_b=0	1	kết quả đại diện
13	Coverage-guided	pin_a=2, pin_b=3390	3392	kết quả đại diện
14	Concolic / white-box (SMT + hằng số)	pin_a=7421, pin_b=3390	EXCEPTION ValueError	kích hoạt lỗi
15	Concolic / white-box (SMT + hằng số)	pin_a=7421, pin_b=3391	10812	kết quả đại diện
16	Concolic / white-box (SMT + hằng số)	pin_a=7422, pin_b=0	7422	kết quả đại diện
17	Concolic / white-box (SMT + hằng số)	pin_a=3389, pin_b=1	3390	kết quả đại diện

Hàm 'parse_atm_pin' (C): 15 test case, 12 kích hoạt lỗi. Coverage hàm 'parse_atm_pin' (probe): 4/4 (100%)

#	Kỹ thuật	Input	Kết quả / lỗi	Ghi chú
1	Biên / giá trị đặc biệt	raw_pin='<script>alert(1)</script>'	TIMEOUT TIMEOUT	kích hoạt lỗi
2	Biên / giá trị đặc biệt	raw_pin='AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'	TIMEOUT TIMEOUT	kích hoạt lỗi
3	Biên / giá trị đặc biệt	raw_pin='AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'	CRASH ACCESS_VIOLATION	kích hoạt lỗi
4	Biên / giá trị đặc biệt	raw_pin='AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA'	CRASH ACCESS_VIOLATION	kích hoạt lỗi

#	Kỹ thuật	Input	Kết quả / lỗi	Ghi chú
5	Biên / giá trị đặc biệt	raw_pin='hello'	None	+4 phần tử coverage mới
6	Black-box (ngẫu nhiên)	raw_pin='78f8v4/4LJpx13p%n WGySnuItP5jkYBgKeC-lON6iGj2zmYy6-EUA.1wAMhTt_8Qrqgvsi	TIMEOUT TIMEOUT	kích hoạt lỗi
7	Black-box (ngẫu nhiên)	raw_pin='EdTV1wjcBzs-qGg7UVKY0VDKbOIpeBVxPLUIw6va0Ey4IwT0% 99.4Vn%Zqn8 mqGEjQYL/	CRASH ACCESS_VIOLATION	kích hoạt lỗi
8	Black-box (ngẫu nhiên)	raw_pin='/uGgPmJrmdfSCrdd0 QSt_t7GST_Jp7I'	TIMEOUT TIMEOUT	kích hoạt lỗi
9	Black-box (ngẫu nhiên)	raw_pin='GA%Xh_U-%pu1Qrjjm5S_rzbKHDurr.ic_3FuG20j/tT.RHtU2gL-l/ds2zMWsn5C_p1nJrp	CRASH ACCESS_VIOLATION	kích hoạt lỗi
10	Black-box (ngẫu nhiên)	raw_pin='Khg%6TYvn9tl RGr1rT/4otMLuU/I7pgJ6gyrm%0.AytRFGH1_zbH%8AVQB1tA2VPksIqX	CRASH STACK_BUFFER_OVERRUN	kích hoạt lỗi
11	Black-box (ngẫu nhiên)	raw_pin='ekmTEcd'	None	kết quả đại diện
12	Mutation (đột biến seed)	raw_pin='AAAA

#	Kỹ thuật	Input	Kết quả / lỗi	Ghi chú
13	Mutation (đột biến seed)	raw_pin='AAAA

Nguồn	Vị trí	Loại phát hiện	CWE	Ví dụ CVE minh họa
ch2	dong 8	dangerous-function	CWE-120 — Buffer Copy without Checking Size of Input	CVE-2003-0352 (2003)
ch2	parse_atm_pin() [wrap]	c-hang	CWE-835 — Loop with Unreachable Exit Condition ('Infinite Loop')	-
ch2	parse_atm_pin() [wrap]	c-out-of-bounds	CWE-787 — Out-of-bounds Write	-
ch3	atm_auth_counter	integer-overflow	CWE-190 — Integer Overflow or Wraparound	CVE-2016-5195 (2016)
ch4	atm_pin_retry_ratio	uncontrolled-crash	CWE-248 — Uncaught Exception	-
ch4	atm_admin_override_pin	uncontrolled-crash	CWE-248 — Uncaught Exception	-
ch5	dong 1	missing-hardening-flags	(không xác định)	-
ch5	dong 1	missing-dynamic-verification	(không xác định)	-

8. Test case truy vết từ đặc tả (tài liệu hóa)

ID	Ưu tiên	Tiêu đề	Truy vết yêu cầu	Kết quả mong đợi
TC-1	HIGH	Xác thực PIN hợp lệ và chuyển trạng thái	REQ-1, REQ-2	Hệ thống chuyển sang trạng thái AUTHENTICATED; công thức EF(AUTHENTICATED) đạt ĐÚNG
TC-2	HIGH	Xử lý nhập sai mã PIN và khóa an toàn	REQ-3, REQ-4	Hệ thống chuyển sang LOCKED; công thức AG(!(LOCKED && AUTHENTICATED)) và EF(LOCKED) đạt ĐÚNG
TC-3	HIGH	Kiểm tra an toàn bộ đệm chuỗi PIN đầu vào	REQ-1, REQ-6	Phát hiện cảnh báo nguy cơ tràn bộ đệm CWE-120 do sao chép chuỗi không giới hạn kích thước

ID	Ưu tiên	Tiêu đề	Truy vết yêu cầu	Kết quả mong đợi
TC-4	MEDIUM	Black-box fuzzing hàm tỷ lệ thử lại mã PIN	REQ-3	Bắt được ngoại lệ ZeroDivisionError (CWE-248) khi số lần sai failed_attempts bằng 0
TC-5	LOW	Khám phá mã PIN quản trị ẩn bằng Concolic Fuzzing	REQ-5	Tìm được chính xác bộ giá trị bí mật (7421, 3390) gây kích hoạt nhánh rẽ ẩn trong <= 15 lần thử
TC-6	MEDIUM	Kiểm tra nguy cơ tràn số nguyên bộ đếm xác thực ATM	REQ-3	Kết quả trả về SAT (phát hiện giá trị có thể gây tràn số nguyên CWE-190)